

TEA

Triagem de Autismo

EDUARDO GIBERTONI CAMILLO
GUSTAVO SCHIZARI FERREIRA FILHO
MARIA CLARA CARDOSO COSTA

Projeto Integrador — 5º Semestre DSM
Versão 1.0 · 2026

Grupo PI5-G04-2026-01

FATEC Franca · Desenvolvimento de Software Multiplataforma

⚠ **AVISO IMPORTANTE:** Este sistema tem caráter exclusivamente informativo e não substitui avaliação clínica realizada por profissional de saúde habilitado. Os resultados do questionário AQ-10 não devem ser interpretados como diagnóstico médico.

1 Introdução

1.1 Objetivo do Documento

Este documento apresenta, de forma técnica e estruturada, a arquitetura, as tecnologias empregadas, os módulos, as funcionalidades e as diretrizes de desenvolvimento da plataforma TEA — Triagem de Autismo. Seu objetivo é servir como referência central para a equipe do projeto e para avaliadores acadêmicos, garantindo visão unificada e atualizada da solução.

1.2 Contexto e Motivação

O Transtorno do Espectro Autista (TEA) afeta milhões de pessoas no Brasil e no mundo, e o diagnóstico tardio compromete o acesso a intervenções eficazes. Ferramentas de triagem informativa, como o questionário AQ-10 (Autism-Spectrum Quotient — 10 itens), permitem que indivíduos e familiares identifiquem características associadas ao TEA e busquem avaliação profissional com mais consciência.

O projeto PI5-G04 foi desenvolvido no contexto do Projeto Integrador do 5º semestre do curso de Desenvolvimento de Software Multiplataforma (DSM) da FATEC Franca como uma solução multiplataforma que integra questionário padronizado, classificação por aprendizado de máquina e localização de clínicas próximas.

1.3 Público-Alvo

- Desenvolvedores: estrutura de código, rotas, padrões e integrações.
- Arquitetos de software: decisões arquiteturais e escalabilidade.
- Avaliadores acadêmicos: visão geral do projeto, tecnologias e entregas.
- Usuários finais (informativo): como acessar e utilizar a plataforma.

2 Visão Geral do Projeto

2.1 Descrição

O TEA é uma plataforma web e mobile de triagem informativa para características associadas ao Transtorno do Espectro Autista. O sistema aplica o questionário AQ-10, classifica as respostas por meio de um modelo de aprendizado de máquina (Random Forest) treinado sobre o Autism Screening Adult Dataset, e exibe clínicas e profissionais próximos ao usuário via integração com OpenStreetMap e Google Maps.

O sistema não realiza diagnóstico clínico. Os resultados são orientativos e têm finalidade exclusivamente informativa, devendo ser interpretados por profissional de saúde.

2.2 Módulos do Sistema

Módulo	Stack	Função	Porta
API de ML (FastAPI)	FastAPI + scikit-learn + Python 3.11	Classificação AQ-10 via Random Forest	8000
API de Usuários (Express)	Express + Prisma + MySQL + Node 18	Cadastro, login e gestão de usuários	8001
Frontend Web (React)	React 19 + Vite 6 + TypeScript 5	Interface web: questionário, resultado, conta	5173
App Mobile (Expo)	React Native + Expo 54 + JavaScript	Aplicativo iOS/Android com os mesmos fluxos	Expo Dev

2.3 Fluxo Principal

1. Usuário acessa o frontend (web ou mobile) e responde ao questionário AQ-10.
2. As respostas e dados complementares (idade, gênero, etnia, histórico familiar) são enviados via POST /predict para a API de ML (porta 8000).
3. A API executa o pipeline de pré-processamento e o classificador Random Forest, retornando classificação (YES/NO) e probabilidades.
4. O frontend exibe o resultado com mensagem orientativa e oferece busca de clínicas próximas via geolocalização do navegador/dispositivo + OpenStreetMap.
5. O cadastro e o login do usuário são opcionais e gerenciados pela API de Usuários (porta 8001) com persistência em MySQL.

3 Tecnologias Utilizadas

3.1 API de Classificação ML (FastAPI)

Tecnologia	Versão	Finalidade
Python	3.11+	Runtime da API de ML
FastAPI	≥ 0.115.0	Framework HTTP assíncrono com documentação automática
Uvicorn	≥ 0.32.0	Servidor ASGI de alta performance
scikit-learn	≥ 1.5.0	Pipeline de ML: imputação, encoding e Random Forest
pandas	≥ 2.2.0	Manipulação e leitura do dataset CSV
joblib	≥ 1.4.0	Serialização e carregamento do modelo treinado
Pydantic	≥ 2.9.0	Validação de schemas de entrada e saída
pydantic-settings	≥ 2.6.0	Configuração via variáveis de ambiente
python-dotenv	≥ 1.0.0	Carregamento do arquivo .env

3.2 API de Usuários (Express + Prisma)

Tecnologia	Versão	Finalidade
Node.js	18+	Runtime JavaScript server-side
Express	^4.21.2	Framework HTTP para API RESTful
TypeScript	^5.7.2	Tipagem estática no backend
Prisma ORM	^6.9.0	Mapeamento objeto-relacional e migrations
MySQL	8+	Banco de dados relacional
bcryptjs	^2.4.3	Hash seguro de senhas
cors	^2.8.5	Controle de acesso entre origens
dotenv	^16.4.7	Carregamento de variáveis de ambiente
tsx	^4.19.2	Execução TypeScript sem transpilação prévia

3.3 Frontend Web (React + Vite)

Tecnologia	Versão	Finalidade
React	^19.0.0	Biblioteca base para construção de interfaces
Vite	^6.0.3	Bundler e servidor de desenvolvimento
TypeScript	~5.7.2	Tipagem estática no frontend
OpenStreetMap / Overpass API	—	Busca de clínicas próximas sem backend
Google Maps (link externo)	—	Fallback para rota até a clínica

3.4 Aplicativo Mobile (React Native + Expo)

Tecnologia	Versão	Finalidade
React Native	^0.81.5	Framework base para apps multiplataforma
Expo	^54.0.20	Plataforma gerenciada para build e testes
JavaScript	—	Linguagem principal do módulo mobile
axios	^1.17.0	Cliente HTTP para chamadas às APIs
@react-navigation	^7.x	Navegação entre telas (Stack + Tab)
AsyncStorage	^2.2.0	Persistência local de sessão
expo-location	~19.0.8	Geolocalização do dispositivo
expo-linear-gradient	^15.0.7	Gradientes visuais
@expo/vector-icons	^15.0.3	Ícones nativos
styled-components	^6.1.19	Estilização por componentes
react-native-chart-kit	^6.12.0	Gráficos e visualizações

3.5 Banco de Dados e Ferramentas Auxiliares

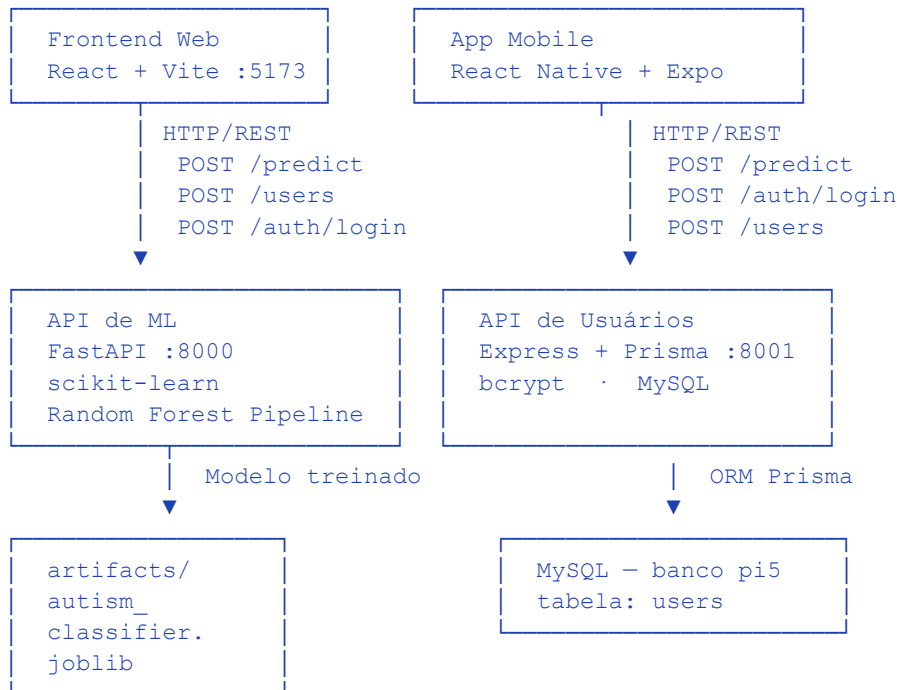
Ferramenta	Finalidade
MySQL 8+	Banco relacional para persistência de usuários
Prisma Schema	Definição da entidade User e migrations
Git + GitHub	Controle de versão e colaboração
Swagger UI (FastAPI /docs)	Documentação interativa da API de ML
OpenStreetMap / Overpass	Geolocalização de clínicas sem custo

4 Arquitetura do Sistema

4.1 Visão Geral

A plataforma adota arquitetura de microsserviços leve, com dois backends independentes convivendo no mesmo repositório, um frontend web e um aplicativo mobile. Cada serviço tem responsabilidade única e se comunica via HTTP/REST.

4.2 Diagrama Descritivo



* Busca de clínicas: frontend → navegador → OpenStreetMap/Overpass (sem backend)

4.3 Comunicação entre Camadas

Origem	Destino	Protocolo	Finalidade
Frontend / Mobile	FastAPI :8000	HTTP POST /predict	Classificação AQ-10
Frontend / Mobile	Express :8001	HTTP POST /users	Cadastro de usuário
Frontend / Mobile	Express :8001	HTTP POST /auth/login	Autenticação
Frontend / Mobile	Express :8001	HTTP GET/PUT/DELETE /users	CRUD de usuários
Browser / Dispositivo	OpenStreetMap / Overpass	HTTP GET (Overpass API)	Clínicas próximas
Browser / Dispositivo	Google Maps	Link externo	Navegação até clínica

Origem	Destino	Protocolo	Finalidade
FastAPI	artifacts/ (disco)	joblib.load()	Carregamento do modelo
Express / Prisma	MySQL :3306	TCP (Prisma ORM)	Persistência relacional

4.4 Padrões Adotados

API de ML (FastAPI)

Padrão	Aplicação
Modular por camada	api/ · models/ · services/ · ml/ separados
Schema-first	Pydantic valida entrada e saída em todos os endpoints
Pipeline scikit-learn	Pré-processamento + classificador em objeto único serializado
Singleton de serviço	prediction_service instanciado uma vez e reutilizado
Lazy loading	Modelo carregado na primeira predição, não na inicialização

API de Usuários (Express)

Padrão	Aplicação
Router segregado	usersRouter e authRouter isolados em arquivos distintos
Lib helpers	password.ts, userFormat.ts, prisma.ts centralizados em lib/
Hash de senha	bcryptjs — nunca expõe password_hash nas respostas
Tratamento de erros Prisma	Códigos P2002 (conflict) e P2025 (not found) tratados

Frontend Web (React)

Padrão	Aplicação
Component-based	Componentes focados: Questionnaire, ResultView, NearbyClinics, etc.
Services layer	api.ts, clinics.ts e users.ts isolam chamadas HTTP
Typed constants	questions.ts exporta AQ_QUESTIONS, ETHNICITY_OPTIONS, RELATION_OPTIONS
Session via localStorage	Chave pi5_user persiste dados do usuário logado

Mobile (React Native)

Padrão	Aplicação
Navegação hierárquica	Stack navigator: Login → Home → Questionario → Resultado / Clinicas
AuthContext	Estado de autenticação compartilhado via React Context

Padrão	Aplicação
Services isolados	api.js, authApi.js, predictionApi.js, clinics.js
IP configurável	URL das APIs definida por constante (necessária para dispositivo físico)

5 Estrutura dos Projetos

5.1 Backend — API de ML (FastAPI)

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py                # Instância FastAPI, CORS, inclusão do router
│   ├── config.py              # Settings via pydantic-settings + .env
│   ├── api/
│   │   ├── __init__.py
│   │   └── routes.py          # GET /health · POST /predict
│   ├── models/
│   │   ├── __init__.py
│   │   └── schemas.py         # PredictRequest, PredictResponse, HealthResponse
│   ├── services/
│   │   ├── __init__.py
│   │   └── prediction_service.py # PredictionService (singleton)
│   └── ml/
│       ├── __init__.py
│       ├── constants.py       # FEATURE_COLUMNS, CATEGORICAL_COLUMNS, etc.
│       ├── preprocessing.py    # build_model_pipeline() - ColumnTransformer + RF
│       └── train.py            # Script de treinamento e serialização
├── data/
│   └── autism_screening-1.csv  # Dataset Pima Autism Screening (adultos)
├── artifacts/                  # Gerado após python -m app.ml.train
│   └── autism_classifier.joblib
├── requirements.txt
└── .env.example
```

5.2 Backend — API de Usuários (Express + Prisma)

```
backend/
├── src/
│   ├── index.ts               # Entry point Express, CORS, routers
│   ├── lib/
│   │   ├── prisma.ts          # Instância global do PrismaClient
│   │   ├── password.ts         # hashPassword(), verifyPassword(), validatePassword()
│   │   └── userFormat.ts       # formatUser() - remove passwordHash da resposta
│   └── routes/
│       ├── users.ts            # CRUD completo em /users
│       └── auth.ts             # POST /auth/login
├── prisma/
│   ├── schema.prisma          # Modelo User (id, email, phone, passwordHash)
│   └── migrations/             # SQL gerado pelo Prisma Migrate
├── package.json
├── tsconfig.json
└── .env.example
```

5.3 Frontend Web (React + Vite)

```
frontend/
├── src/
│   ├── App.tsx                # Orquestrador de estado e rotas de views
│   └── main.tsx               # Entry point React
```

```

├── App.css / index.css # Estilos globais
├── components/
│   ├── LandingPage.tsx # Página inicial com CTA
│   ├── Questionnaire.tsx # Formulário AQ-10 completo
│   ├── Questionnaire.css
│   ├── AnalysisLoading.tsx # Tela de loading durante classificação
│   ├── AnalysisLoading.css
│   ├── ResultView.tsx # Exibição do resultado
│   ├── NearbyClinics.tsx # Busca de clínicas por geolocalização
│   ├── NearbyClinics.css
│   ├── AccountLogin.tsx # Tela de login
│   ├── AccountRegister.tsx # Tela de cadastro
│   ├── AccountRegister.css
│   └── Layout.tsx # Wrapper com header e navegação
├── constants/
│   ├── questions.ts # AQ_QUESTIONS, ETHNICITY_OPTIONS, RELATION_OPTIONS
│   └── brand.ts # Nome e metadados do projeto
├── services/
│   ├── api.ts # fetchHealth(), predict() → FastAPI :8000
│   ├── clinics.ts # findNearbyMentalHealthClinics() → OpenStreetMap
│   └── users.ts # createUser(), login() → Express :8001
├── types/
│   ├── predict.ts # PredictPayload, PredictResponse
│   └── user.ts # User
├── utils/
│   ├── accountSession.ts # Leitura/escrita de pi5_user no localStorage
│   └── phoneMask.ts # Máscara de telefone
├── public/
│   └── logo.png
├── index.html
├── vite.config.ts
├── package.json
└── .env.example

```

5.4 Aplicativo Mobile (React Native + Expo)

```

mobile/
├── src/
│   ├── components/
│   │   ├── BottomNavCustom.js # Barra de navegação inferior
│   │   ├── HeaderCustom.js # Cabeçalho reutilizável
│   │   ├── QuestionarioCard.js # Card individual de pergunta
│   │   └── ResultadoCard.js # Card de exibição do resultado
│   ├── constants/
│   │   ├── questions.js # Perguntas AQ-10 em português
│   │   └── countries.js # Lista de países
│   ├── context/
│   │   └── AuthContext.js # Estado global de autenticação
│   ├── pages/
│   │   ├── Home.js # Tela inicial
│   │   ├── Login.js # Autenticação
│   │   ├── Cadastro.js # Registro de novo usuário
│   │   ├── Questionario.js # Questionário AQ-10
│   │   ├── Resultado.js # Exibição da classificação
│   │   └── Clinicas.js # Busca de clínicas próximas

```

```
|
| | └─ services/
| | |   └─ api.js           # Configuração base do axios
| | |   └─ authApi.js       # login/cadastro → Express :8001
| | |   └─ predictionApi.js # predict() → FastAPI :8000
| | |   └─ clinics.js       # Busca OpenStreetMap + expo-location
| | |
| | └─ utils/
| | |   └─ getUsuario.js     # Leitura do usuário em AsyncStorage
| | |   └─ storage.js        # Helpers de AsyncStorage
| | |   └─ phoneMask.js      # Máscara de telefone
| | |
| | └─ routes.js            # Definição da navegação (Stack + Tab)
| | └─ styles.js            # Design system centralizado
└─ assets/
  └─ logo.png
└─ App.js
└─ app.json
└─ babel.config.js
└─ package.json
```

6 Módulo de Machine Learning

6.1 Dataset

O modelo foi treinado sobre o Autism Screening Adult Dataset, derivado do Autism-Spectrum Quotient (AQ-10) aplicado a adultos. O arquivo CSV `autism_screening-1.csv` contém registros com 10 perguntas binárias (A1–A10), metadados demográficos e o rótulo-alvo `Class/ASD` (YES/NO).

Campo	Tipo	Descrição
A1_Score ... A10_Score	int (0 ou 1)	Respostas binárias do questionário AQ-10
age	float	Idade do respondente em anos
gender	str (m/f)	Gênero
ethnicity	str	Etnia declarada
jundice	str (yes/no)	Icterícia neonatal
austim	str (yes/no)	Histórico familiar de autismo
contry_of_res	str	País de residência
used_app_before	str (yes/no)	Uso anterior de app de triagem
result	float (0–10)	Soma calculada de A1–A10
age_desc	str	Faixa etária (dataset: '18 and more')
relation	str	Quem preencheu (Self, Parent, etc.)
Class/ASD	str (YES/NO)	Rótulo-alvo (variável dependente)

6.2 Pipeline de Pré-processamento

O pipeline é construído com `ColumnTransformer` e combina dois ramos:

- Colunas numéricas (A1–A10, age, result): imputação pela mediana via `SimpleImputer`.
- Colunas categóricas (gender, ethnicity, jundice, austim, contry_of_res, used_app_before, age_desc, relation): imputação pela moda + `OneHotEncoder` com `handle_unknown='ignore'`.

O pipeline completo é serializado como um único objeto `joblib`, garantindo que o mesmo pré-processamento utilizado no treino seja aplicado exatamente na inferência.

6.3 Classificador

Parâmetro	Valor
Algoritmo	<code>RandomForestClassifier</code> (scikit-learn)
n_estimators	100 árvores
random_state	42 (reprodutibilidade)
class_weight	balanced (lida com desequilíbrio de classes)

Parâmetro	Valor
n_jobs	-1 (paralelismo total)
test_size	0.2 (20% para avaliação)
Estratégia de split	Estratificado (stratify=y) para manter proporção YES/NO

6.4 Treinamento e Serialização

O script de treinamento é executado via:

```
python -m app.ml.train
```

Ao final, exibe acurácia no conjunto de teste e relatório de classificação (precision, recall, F1 por classe). O modelo é salvo em `artifacts/autism_classifier.joblib` e carregado sob demanda pela `PredictionService`.

6.5 Inferência

A `PredictionService` implementa lazy loading: o modelo é carregado na primeira chamada a `predict()` e reutilizado nas subsequentes. O método retorna:

- `classification`: "YES" ou "NO"
- `probability`: confiança na classe predita (0.0 a 1.0)
- `probabilities`: dicionário com probabilidades de ambas as classes
- `message`: mensagem orientativa em português

7 Banco de Dados

7.1 Modelo de Dados

A camada de persistência utiliza MySQL 8+ acessado via Prisma ORM. No estado atual, o banco contém a entidade User, responsável pelo controle de acesso à plataforma.

7.2 Entidade User

Campo	Tipo	Restrições	Descrição
id	Int	PK, autoincrement	Identificador único
email	VarChar(255)	UNIQUE, NOT NULL	E-mail de login
phone	VarChar(20)	NOT NULL	Telefone de contato
password_hash	VarChar(255)	NOT NULL	Hash bcrypt da senha
created_at	DateTime	DEFAULT now()	Data de criação
updated_at	DateTime	Auto-atualizado	Data da última edição

7.3 Configuração

```
DATABASE_URL="mysql://USUARIO:SENHA@localhost:3306/pi5"
```

```
-- Criar o banco antes da migration:
```

```
CREATE DATABASE pi5 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

7.4 Scripts Prisma

Comando	Descrição
npm run db:migrate	Aplica migrations — necessário na primeira execução
npm run db:generate	Regenera o client Prisma após mudanças no schema
npm run db:studio	Abre interface visual do banco (Prisma Studio)
npm run db:push	Sincroniza schema sem criar migration (dev rápido)

8 Funcionalidades Principais

8.1 Questionário AQ-10

Implementado tanto no frontend web quanto no app mobile, com as mesmas 10 perguntas padronizadas do Autism-Spectrum Quotient:

#	Pergunta
1	Costumo notar pequenos sons quando estou em um ambiente barulhento.
2	Costumo concentrar-me mais em todo o conjunto do que nos pequenos detalhes.
3	Tenho dificuldade em fazer mais do que uma coisa de cada vez.
4	Se houver uma interrupção, posso retomar facilmente o que estava a fazer.
5	Acho fácil ler emoções nas pessoas através da expressão facial.
6	Sei dizer se alguém que escuta a uma conversa está aborrecido ou entediado.
7	Quando leio uma história, tenho dificuldade em perceber as intenções dos personagens.
8	Gosto de recolher informações sobre categorias de coisas.
9	Acho fácil imaginar como seria ser outra pessoa.
10	Acho difícil fazer amizades com outras pessoas.

8.2 Dados Complementares do Formulário

Campo	Tipo	Opções
Idade	Numérico	1 a 120 anos
Gênero	Seleção	Masculino (m) / Feminino (f)
Etnia	Seleção	Branco-Europeu, Latino, Asiático, Negro, Oriente Médio, Outro
País de residência	Texto livre	Ex.: Brazil
Quem responde	Seleção	Eu mesmo, Pai/Mãe, Parente, Profissional de saúde
Icterícia neonatal	Sim/Não	—
Histórico familiar de autismo	Sim/Não	—
Uso anterior de app de triagem	Sim/Não	—

8.3 Classificação e Resultado

- A API retorna classificação YES (indicativo positivo) ou NO.
- Probabilidade de confiança exibida ao usuário com mensagem orientativa.
- Resultado não requer conta — qualquer visitante pode responder o questionário.

8.4 Busca de Clínicas Próximas

- Acionada a partir da tela de resultado.

- Utiliza API de geolocalização do navegador/dispositivo (Geolocation API / expo-location).
- Consulta Overpass API (OpenStreetMap) por estabelecimentos de saúde mental em raio próximo.
- Exibe nome, distância e endereço de cada clínica encontrada.
- Oferece link direto para abertura de rota no Google Maps (fallback para qualquer resultado).

8.5 Autenticação e Conta (Opcional)

- Cadastro com e-mail, telefone e senha (mínimo 6 caracteres).
- Login com e-mail e senha — retorna dados do usuário sem o hash.
- Sessão mantida em localStorage (web) ou AsyncStorage (mobile).
- Logout limpa a sessão localmente.
- A conta não é necessária para acessar o questionário — serve para identificar participação no projeto.

9 Referência de Endpoints

9.1 API de ML (FastAPI — porta 8000)

Método	Rota	Descrição	Auth
GET	/health	Status da API e se o modelo está carregado	Não
POST	/predict	Classificação AQ-10 — retorna YES/NO + probabilidades	Não
GET	/docs	Swagger UI interativo	Não

Exemplo de payload POST /predict:

```
{
  "A1_Score": 1,  "A2_Score": 0,  "A3_Score": 1,
  "A4_Score": 0,  "A5_Score": 1,  "A6_Score": 0,
  "A7_Score": 1,  "A8_Score": 0,  "A9_Score": 1,  "A10_Score": 0,
  "age": 25,
  "gender": "m",
  "ethnicity": "Latino",
  "jundice": "no",
  "austim": "no",
  "country": "Brazil",
  "used_app_before": "no",
  "age_desc": "18 and more",
  "relation": "Self"
}
```

Exemplo de resposta:

```
{
  "classification": "YES",
  "probability": 0.8700,
  "probabilities": { "NO": 0.1300, "YES": 0.8700 },
  "message": "Indicativo de triagem positiva para TEA (autismo)."
}
```

9.2 API de Usuários (Express — porta 8001)

Método	Rota	Descrição	Auth
GET	/health	Status do serviço	Não
POST	/users	Cadastrar usuário (email, phone, password)	Não
GET	/users	Listar todos os usuários	Não
GET	/users/:id	Buscar usuário por ID	Não
PUT	/users/:id	Atualizar usuário	Não
DELETE	/users/:id	Excluir usuário	Não
POST	/auth/login	Login — retorna dados do usuário (sem hash)	Não

Observação: A API de usuários não implementa JWT no estado atual. A autenticação é validada comparando a senha com o hash bcrypt e retornando os dados do usuário, que são armazenados localmente no cliente (localStorage / AsyncStorage). A implementação de tokens JWT é listada como evolução futura.

9.3 Respostas de Erro Padronizadas

Status HTTP	Condição	Exemplo de body
400 Bad Request	Campos inválidos ou ausentes	{ "detail": "Informe um e-mail válido." }
401 Unauthorized	Credenciais inválidas no login	{ "detail": "E-mail ou senha incorretos." }
404 Not Found	Usuário não encontrado	{ "detail": "Usuário não encontrado." }
409 Conflict	E-mail já cadastrado	{ "detail": "Este e-mail já está cadastrado." }
500 Internal Server Error	Erro inesperado	{ "detail": "Erro interno ao processar a solicitação." }
503 Service Unavailable	Modelo ML não carregado	{ "detail": "model nao encontrado... Execute: python -m app.ml.train" }

10 Segurança

10.1 Proteção de Senhas

- Senhas armazenadas exclusivamente como hash bcrypt via bcryptjs.
- O campo password_hash nunca é retornado nas respostas da API (filtrado por formatUser()).
- Validação mínima de 6 caracteres na senha antes do hash.

10.2 Validação de Entradas

- API de ML: todas as entradas validadas por schemas Pydantic com restrições explícitas (ge=0, le=1 para scores; ge=0, le=120 para idade; Literal para campos enumerados).
- API de Usuários: funções validateEmail(), validatePhone() e validatePassword() aplicadas antes de qualquer operação no banco.
- Strings têm limite de tamanho e são tratadas com strip antes do processamento.

10.3 CORS

- Ambas as APIs configuram CORS via variável de ambiente CORS_ORIGINS.
- Em desenvolvimento: origins incluem http://localhost:5173.
- Configuração permissiva (*) disponível para facilitar desenvolvimento, devendo ser restringida em produção.

10.4 Boas Práticas Adotadas

Prática	Implementação
Hash de senha	bcrypt — nunca plain text
Separação de responsabilidades	password.ts e userFormat.ts isolam lógica de segurança
Tratamento de erros Prisma	Códigos P2002 e P2025 retornam mensagens seguras sem expor stack
Variáveis de ambiente	.env não commitado — .env.example fornecido como template
Sem JWT atual	Sessão local — evolução para tokens planejada

11 Deploy e Infraestrutura

11.1 Pré-requisitos

Requisito	Versão mínima	Uso
Python	3.11+	API de ML e treinamento do modelo
Node.js + npm	18+	API de usuários e frontend
MySQL	8+	Banco de dados relacional
Expo CLI	Qualquer recente	Build e execução do app mobile

11.2 Configuração Inicial (uma vez)

Banco de dados

```
-- Execute no MySQL:  
CREATE DATABASE pi5 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Backend

```
cd backend  
copy .env.example .env          # Windows  
# cp .env.example .env          # Linux/macOS  
  
# Edite .env e configure:  
DATABASE_URL="mysql://USUARIO:SENHA@localhost:3306/pi5"  
  
# Ambiente Python (API de ML):  
python -m venv .venv  
.\.venv\Scripts\Activate.ps1    # Windows PowerShell  
# source .venv/bin/activate      # Linux/macOS  
pip install -r requirements.txt  
python -m app.ml.train          # Treina e salva o modelo  
  
# API de usuários (Node):  
npm install  
npm run db:migrate
```

Frontend

```
cd frontend  
copy .env.example .env  
npm install
```

11.3 Execução Diária (3 terminais)

Termin al	Pasta	Comando	Resultado
1	backend/	.venv\Scripts\Activate.ps1 && uvicorn app.main:app --reload --host 0.0.0.0 --port 8000	API de ML em :8000

Terminal	Pasta	Comando	Resultado
2	backend/	npm run dev	API de usuários em :8001
3	frontend/	npm run dev	Frontend em :5173

Acesse a aplicação em: <http://localhost:5173>

11.4 Execução do App Mobile

```
cd mobile
npm install
npx expo start
```

```
# No emulador Android: pressione 'a'
# No navegador web: pressione 'w'
# Em dispositivo físico: escaneie o QR Code com o app Expo Go
```

Importante (dispositivo físico): substitua localhost pelo IP da máquina host nas constantes de URL em `src/services/authApi.js` e `src/services/predictionApi.js`. Use `ipconfig` (Windows) ou `ifconfig` (Linux/macOS) para obter o IPv4 da rede local. Exemplo: <http://192.168.0.100:8000>

11.5 Ambientes

Ambiente	Finalidade	URL
Development (local)	Desenvolvimento e testes	http://localhost:5173
API de ML (local)	Classificação e health check	http://localhost:8000
API de ML — Swagger	Documentação interativa	http://localhost:8000/docs
API de Usuários (local)	Cadastro e login	http://localhost:8001
Mobile (Expo Dev Server)	Build e hot reload mobile	Expo QR Code

11.6 Versionamento e Branches

Branch	Finalidade
main	Código principal do projeto
feature/*	Desenvolvimento de novas funcionalidades
fix/*	Correção de bugs

12 Requisitos do Sistema

12.1 Requisitos Funcionais

ID	Requisito	Módulo	Prioridade
RF01	O sistema deve aplicar o questionário AQ-10 completo com 10 perguntas binárias	Web / Mobile	Alta
RF02	O sistema deve coletar dados complementares: idade, gênero, etnia, país, relação, histórico familiar	Web / Mobile	Alta
RF03	A API deve classificar as respostas em YES/NO com probabilidade de confiança	API ML	Alta
RF04	O resultado deve exibir mensagem orientativa e não diagnóstica	Web / Mobile	Alta
RF05	O usuário deve poder buscar clínicas próximas após ver o resultado	Web / Mobile	Alta
RF06	A busca de clínicas deve usar a geolocalização do dispositivo do usuário	Web / Mobile	Alta
RF07	O sistema deve permitir cadastro com e-mail, telefone e senha	API Usuários	Média
RF08	O sistema deve permitir login com e-mail e senha	API Usuários	Média
RF09	O questionário deve funcionar sem necessidade de conta/login	Web / Mobile	Alta
RF10	A API de ML deve oferecer endpoint /health indicando se o modelo está carregado	API ML	Média

12.2 Requisitos Não Funcionais

ID	Requisito	Categoria
RNF01	A API de ML deve responder a /predict em menos de 2 segundos para 95% das requisições	Performance
RNF02	Senhas devem ser armazenadas como hash bcrypt — nunca em plain text	Segurança
RNF03	O campo password_hash nunca deve ser exposto nas respostas da API	Segurança
RNF04	O frontend deve funcionar nos principais navegadores modernos (Chrome, Firefox, Edge)	Compatibilidade
RNF05	O app mobile deve funcionar em Android 10+ e iOS 15+	Compatibilidade
RNF06	O sistema deve validar todas as entradas da API antes de processar	Confiabilidade
RNF07	O modelo de ML deve ser serializado e carregado sem re-treinamento a cada requisição	Performance
RNF08	As variáveis de ambiente sensíveis não devem ser commitadas no repositório	Segurança

13 Integrações Externas

13.1 OpenStreetMap / Overpass API

Utilizada para localizar clínicas e profissionais de saúde mental próximos ao usuário. A consulta é realizada diretamente pelo navegador (frontend), sem intermediação de backend, utilizando a Overpass API pública.

Aspecto	Detalhe
URL base	https://overpass-api.de/api/interpreter
Query	Busca por amenity=clinic, amenity=hospital e healthcare=* em raio configurável
Dados retornados	Nome, coordenadas e endereço (quando disponível) dos estabelecimentos
Limite de distância	Calculado em km com base nas coordenadas do usuário
Fallback	Link Google Maps com busca por clínicas próximas ao ponto do usuário
Custo	Gratuito — sem necessidade de chave de API

13.2 Google Maps (link externo)

Utilizado como fallback e complemento à busca de clínicas. O frontend gera uma URL de busca direta no Google Maps com base nas coordenadas do usuário. O link abre em nova aba — sem uso de API Key ou SDK.

13.3 expo-location (Mobile)

Biblioteca Expo para obtenção da localização GPS do dispositivo. Solicita permissão explícita ao usuário antes de acessar a posição. Usada na tela Clínicas do app mobile para o mesmo fluxo de busca via OpenStreetMap.

13.4 Integrações Futuras

Integração	Justificativa	Prazo estimado
JWT para autenticação stateless	Substituir sessão local por tokens seguros com expiração	Curto prazo
HTTPS em produção	TLS obrigatório para dados em trânsito	Antes do deploy
API de e-mail (ex.: SendGrid)	Envio de confirmação de cadastro e resultados por e-mail	Médio prazo
Histórico de triagens por usuário	Persistir resultados vinculados ao usuário logado	Médio prazo
Exportação de relatório PDF	Permitir que o usuário salve o resultado da triagem	Longo prazo
Integração com modelo analítico mais robusto	Refinar classificação com datasets maiores	Longo prazo

14 Resolução de Problemas

Sintoma	Causa Provável	Solução
Questionário não classifica / erro 503	API de ML offline ou modelo não treinado	Verificar Terminal 1 (FastAPI rodando). Executar <code>python -m app.ml.train</code> se modelo ausente.
Erro ao criar conta ou entrar	API de usuários offline ou banco não configurado	Verificar Terminal 2 (<code>npm run dev</code>). Conferir <code>DATABASE_URL</code> no <code>.env</code> . Confirmar que a migration foi aplicada.
'Este e-mail já está cadastrado'	Usuário já existe no banco	Usar outro e-mail ou fazer login com o existente.
'E-mail ou senha incorretos'	Credenciais inválidas	Conferir e-mail e senha. Usar cadastro se ainda não tiver conta.
App mobile não conecta às APIs	localhost não acessível em dispositivo físico	Substituir localhost pelo IP da máquina na rede local nos arquivos <code>authApi.js</code> e <code>predictionApi.js</code> .
Clínicas não aparecem na busca	Permissão de localização negada ou Overpass sem resultados na região	Conceder permissão de localização. Usar o link Google Maps como alternativa.
Erro ao rodar <code>npm run db:migrate</code>	Banco pi5 não existe ou <code>DATABASE_URL</code> incorreto	Criar banco: <code>CREATE DATABASE pi5...</code> Verificar credenciais no <code>.env</code> .
Modelo não encontrado após treinamento	<code>MODEL_PATH</code> incorreto no <code>.env</code>	Verificar caminho em <code>MODEL_PATH=artifacts/autism_classifier.joblib</code>

15 Considerações Finais

15.1 Pontos Fortes

Aspecto	Destaque
Arquitetura modular	Dois backends independentes com responsabilidades claras facilitam manutenção e evolução isolada
Pipeline ML completo	Pré-processamento + classificador em um único artefato serializado garante consistência entre treino e inferência
TypeScript end-to-end	Tipagem estática em Express e React elimina erros em runtime e melhora a manutenibilidade
Validação robusta	Pydantic no Python e funções de validação dedicadas no Node garantem integridade dos dados
Multiplataforma real	Web (React) e mobile (React Native/Expo) com os mesmos fluxos e integração com as mesmas APIs
Busca de clínicas sem custo	OpenStreetMap/Overpass API sem necessidade de chave ou contrato com fornecedor
Questionário padronizado	AQ-10 internacionalmente reconhecido, localizado em português com clareza

15.2 Melhorias Futuras

Curto Prazo (1–3 meses)

Melhoria	Justificativa
Autenticação JWT	Substituir sessão local por tokens com expiração e segurança aprimorada
HTTPS em produção	Proteger dados em trânsito — obrigatório antes de qualquer deploy público
Histórico de triagens por usuário	Permitir que usuários acompanhem evolução das respostas ao longo do tempo
Testes automatizados	Ampliar cobertura com pytest (ML) e Jest (Node/React) para estabilidade nas entregas

Médio Prazo (3–6 meses)

Melhoria	Justificativa
Exportação de resultado em PDF	Usuário pode compartilhar resultado com profissional de saúde
Notificações por e-mail	Confirmação de cadastro e envio do resultado por e-mail
Internacionalização (i18n)	Suporte a inglês e espanhol para ampliar alcance
Dark mode	Conforto visual em ambientes com baixa luminosidade

Longo Prazo (6–12 meses)

Melhoria	Justificativa
Modelo aprimorado com dataset maior	Melhorar acurácia e generalização do classificador
Perfil profissional de saúde	Módulo separado para profissionais acompanharem pacientes
Integração com prontuário eletrônico	Interoperabilidade com sistemas de saúde via FHIR
Suporte ao questionário infantil (AQ-10 Child)	Ampliar escopo para triagem em crianças